Caderno de Pesquisa — Memphis-V Graphical Debugger (Python)

Nov 12, 2026 · @Guilherme Rosmann

Informações gerais

Este caderno registra a refatoração do Graphical Debugger do Memphis-V de Java para Python/PySide6, de 02/09/26 a 12/11/26: da primeira linha de código à versão 1.0.0, ao pitch e à apresentação final.

Item	Descrição
Projeto	Memphis-V Graphical Debugger (Python/PySide6)
Objetivo	Reescrever em Python o GraphicalDebugger original em Java do framework Memphis-V, que simula MPSoCs com NoC, com interface moderna, manutenção mais simples e execução sem JVM
Disciplina	Prática em Pesquisa 2026/2
Equipe	Guilherme de Lima Rosmann (GitHub: DrakRosmann), Matheus Lopes Azambuja, Laura Barcelos Rosa, Max Weiss Breuel
Orientador	Prof. Dr. Fernando Gehm Moraes
Repositório	github.com/DrakRosmann/ProjectMemphisPy, branch `Develop` (integração na `main` por pull request)
Ferramenta de apoio	Claude Code (assistente de programação), usado a partir de 23/09/26 em 9 sessões de desenvolvimento
Artigo de referência	Ruaro et al., *Memphis: a framework for heterogeneous many-core SoCs generation and validation*, DAES 2019

Convenções deste caderno

- Cada entrada tem a data no formato dd/mm/yy, seguida do que foi feito, das decisões tomadas e dos próximos passos.
- Horários estão no fuso de Brasília (UTC-3).

- As entradas de 02/09/26 a 21/09/26 foram reconstruídas a partir do histórico de commits do Git, porque não há sessões do Claude registradas nesse período.
- As entradas de 23/09/26 a 29/09/26 vêm do histórico de commits e das conversas com o Claude Code; as de outubro, do Git e do arquivo do pitch. Pedidos feitos ao assistente aparecem entre aspas.
- Itens concluídos em "Próximos passos" aparecem riscados nas entradas seguintes, como em um caderno físico.
- Commits seguem o padrão do README: mensagem terminando em `[feature]`, `[enhancement]` ou `[bugfix]`.

Ambiente e comandos usados

O projeto roda com Python 3.14 e PySide6 ≥ 6.7, gerenciado pelo `uv`; o desenvolvimento foi feito no Fedora (GNOME/Wayland) com PyCharm.

Instalar e executar

```
uv sync --locked
uv run main.py
```

Rodar os testes (pytest, sem tela; usam o cenário do `DEBUG EXAMPLE.zip`)

```
uv run --locked pytest -q
```

Gerar o executável Linux (mesmo comando do CI em `.github/workflows/build.yml`; o resultado fica em `dist/MemphisVDebugger/`, não em `build/`)

```
uv run --locked --with pyinstaller pyinstaller --noconfirm --clean --onedir --windowed --name MemphisVDebugger --add-data "images:images" --add-data "icon:icon" --add-data "color_spectrum.png:." main.py
```

No Windows (PowerShell) o separador do `--add-data` é `;` e a quebra de linha é a crase.

Abrir o cenário de exemplo

1. Extrair o `DEBUG EXAMPLE.zip`.
2. No debugger, File → New Debugging e escolher `LOGS/example_testcase/example_scenario/debug/`.
3. A pasta precisa ter `platform.cfg`, `services.cfg` e `traffic_router.txt`; sem eles aparece o erro "Invalid Directory".

Gerar e simular um testcase no Memphis-V (servidor do laboratório, via `ssh -p 8888`)

```
~/Memphis-5/bin/memphi5 testcase tc.yaml
~/Memphis-5/bin/memphi5 applications tc app.yaml
~/Memphis-5/bin/memphi5 scenario tc sc.yaml
~/Memphis-5/bin/memphi5 simulate tc/sc --nogui
```

O `memphi5` deve ser o original de `~/Memphis-5/bin`: uma cópia em outra pasta falha com `No module named 'testcase'`. Para copiar arquivos ao servidor, `scp` usa `-P` maiúsculo para a porta.

Hipóteses e perguntas de pesquisa

Quatro hipóteses guiaram o trabalho; três foram confirmadas por experimento e uma foi confirmada em parte.

#	Hipótese	Como foi testada	Resultado
H1	A versão Python reproduz os mesmos resultados do debugger Java para o mesmo trace	Emulação da lógica Java pacote a pacote (inclusive `float` de 32 bits) sobre os 3101 pacotes do exemplo (25/09/26)	Confirmada: tempos, percentuais e pacote atual são iguais. As setas vermelhas diferem por dois bugs do Java, corrigidos no Python
H2	Python, mesmo sendo mais lento como linguagem, processa o trace tão rápido quanto o Java	Mesmo processamento sem interface em um trace de 930.300 pacotes (25/09/26)	Confirmada: Python 7,07 s contra Java 7,18 s. O desenho da interface não foi medido
H3	As métricas e funções que o artigo do Memphis atribui ao debugger podem ser calculadas só com os arquivos que a ferramenta já lê	Implementação de latência, vazão, linha do tempo de admissão e validação XY, conferidas no exemplo (25/09/26)	Confirmada em parte: a linha do tempo precisa também do log do `mapper_task`, porque o Memphis-V não usa diretamente na NoC as mensagens do protocolo do artigo
H4	O número de arquivos Python pode cair sem afetar funcionamento e desempenho	União de módulos e execução da suíte de testes antes e depois (28/09/26)	Confirmada: de 31 para 21 arquivos, 69 de 69 testes passando

No pitch de outubro/26 a equipe formulou a pergunta de pesquisa da etapa seguinte: como tratar e armazenar os eventos de simulação para que o debugger escale com o tamanho do many-core? Ela ainda não tem resultados; o benchmark da H2 é o primeiro dado sobre ela.

Registros diários

02/09/26

Criado o repositório com o esqueleto da aplicação em PySide6 (commit `6973cb6`, "Initial commit"). A interface foi desenhada no Qt Designer e convertida em Python.

- Arquivos: `Menphisui.ui` e `ui_mainwindow.py` (janela principal), `Comm.ui` e `commsUi.py` (Communication Overview), `main.py`, `pyproject.toml`.
- Configuração do projeto no PyCharm (`.idea/`).

Próximos passos:

- Desenhar a janela de Task Mapping.
- Ler o arquivo de configuração da plataforma.

03/09/26

Interface ampliada e primeira leitura de dados do Memphis-V.

- 08:12 — Criada a janela Task Mapping (`taskmap.py`) e ajustadas a principal e a de comunicação (`aa91aab`).
- 08:30 — README inicial com o objetivo do projeto (`9b87ad5`).
- 09:23 — Copiadas do projeto Java as imagens dos roteadores e das setas (`images/`) e os ícones (`icon/`); criados `Roteador.ui` e `taskmap.ui` (`fee6bc4`).
- 20:39 — Criado `util/MPSoCConfig.py`, que lê o `platform.cfg`: dimensões da malha, clusters, clock e tarefas (`5e66808`, `aae8416`).

Próximos passos:

- ~~Ler o arquivo de configuração da plataforma.~~
- Montar a lista de tarefas e a grade de roteadores.

10/09/26

Adicionada a Task List e uma primeira versão da janela do roteador.

- `Roteador.py` criado (`f053e54`) e depois removido (`4c16fac`), para ser refeito.
- Menu Task List mostra nomes e IDs das tarefas lidos pelo `MPSoCConfig` (`f8873ed`).

18/09/26

Adicionado ao repositório o `DEBUG EXAMPLE.zip` (78 MB), um cenário real gerado pelo Memphis-V que passou a ser a base de todos os testes (`665f583`). Implementado o modo escuro (`1ec627b`).

21/09/26

Grade de roteadores montada a partir do `platform.cfg` .

- 08:45 — Criado `router_matrix.py` : grade com o tamanho `mpsoc_x × mpsoc_y` , levando em conta os clusters (`e42aded`).
- 08:50 — Grade adaptada aos temas claro e escuro (`9f687dc`).
- 09:13 — Matrizes de PEs nas janelas Task Mapping e Communication Overview (`pe_matrix.py` , `slave_matrix.py` ; `a1ed248`).
- 16:42 — Max Breuel adicionou ao README o fluxo de contribuição com Git e criou o workflow de build no GitHub Actions, que roda em pull requests (`2b328fa` , `125c38c` ; PR #1).

Próximos passos:

- ~~Montar a lista de tarefas e a grade de roteadores.~~
- Fazer a simulação do tráfego sobre a grade.
- Limpar o código antes de crescer.

23/09/26

Primeira sessão com o Claude Code: revisão e limpeza do código existente antes de seguir com a simulação.

- Pedi uma explicação do `main.py` . O assistente apontou import duplicado do `MPSoCConfig` , imports sem uso e nomes com erro de digitação.
- Pedido: "do this corrections with others files too". Correções em `main.py` , `Roteador.py` e `util/MPSoCConfig.py` :
  - renomeados `updadeSlide` → `update_slide` , `CheckB` → `on_checkbox_changed` , classe `taskMap` → `TaskMapWindow` , constantes `WEAST_*` → `WEST_*` ;
  - removido `getTaskNameHash()` , duplicado de `get_task_name_hash()` ;
  - removida a dependência errada `SortedDict` do `pyproject.toml` e fixado `pyside6>=6.7` (necessário para `checkStateChanged`).
- Os arquivos gerados pelo Qt Designer (`ui_mainwindow.py` , `commsUi.py` , `taskmap.py`) não foram editados, porque são recriados a partir dos `.ui` .
- Teste: a aplicação abriu as três janelas em execução sem tela (modo offscreen).

- Commits `73da216` (limpeza) e `c594d15` (`.gitignore` para `__pycache__/` e `.venv/`), enviados para a `Develop`.

24/09/26

Dia mais produtivo do projeto: a simulação do tráfego da NoC passou a funcionar e foram portadas as janelas de análise. Ao fim do dia foi publicada a release `v0.1.0-alpha`.

Abertura de debug (09:02)

- Mudei a abertura para escolher uma pasta em vez de um `.cfg`. O assistente identificou que, sem `platform.cfg`, o erro só era impresso e a grade era montada com valores padrão.
- Pedido: mostrar uma caixa de diálogo quando a pasta não tiver `platform.cfg`, `services.cfg` e `traffic_router.txt`. Implementado com `REQUIRED_FILES` e `QMessageBox.critical` ("Invalid Directory"). Nada é carregado se faltar um arquivo.
- Observação: no exemplo, a pasta certa é `LOGS/example_testcase/example_scenario/debug/`; `LOGS/example_testcase/debug/` não tem o `traffic_router.txt`.

Services List (10:56)

- Tabela igual à da Task List, lida do `services.cfg` (42 serviços, de `NEW_APP 0` a `ANNOUNCE_MONITOR 83`). As duas janelas usam a mesma função `create_table_window`.
- Erro meu e do assistente, corrigido às 16:16: os códigos tinham sido lidos como hexadecimais, mas o Java e o `traffic_router.txt` usam decimal.

Port das classes Java de informação (16:13)

Pedido: "refatore, implemente e junte esses códigos em Java para Python". Cada classe Java virou um módulo no pacote `information/`:

Python	Java
`packet_information.py`	`PacketInformation`
`task_information.py`	`TaskInformation`
`router_neighbors.py`	`RouterNeighbors`
`router_information.py`	`RouterInformation`
`mpsoc_information.py`	`MPSoCInformation`
`port_information.py`	`PortInformation`

Python	Java
`read_traffic_data.py`	`ReadTrafficData`

- Teste: os 3101 pacotes do exemplo foram lidos sem erro; totais de volume, banda e tarefas alocadas conferem.
- Bug do Java encontrado: `getChannel` verificava `EAST0` duas vezes e esquecia `WEST0`, então a porta WEST 0 era marcada como canal LOW. Corrigido: toda porta par é HIGH.

Simulação dos roteadores (16:17–16:37)

- Novo `simulation.py` com `SimulationController` e `CheckpointController`. No Java era uma thread com `sleep()`; aqui virou `QTimer`, porque só a thread principal do Qt pode mexer na interface.
- Controles: `>||` avança um pacote, `>` play, `STOP`, slider de velocidade (99–100 processa em lotes sem redesenhar), Back To (voltar a um tempo em ticks) e Reset Simulation.
- A grade passou a desenhar a imagem de cada roteador e as setas; o caminho do pacote fica vermelho e o uso dos enlaces é recalculado a cada janela de 0,5 ms.
- Caixa Speed Control igual à do original: tempo em ms e em ticks (ex.: `2.48666 ms` / `248666 ticks`).
- Bugs do Java corrigidos: caminhos vermelhos nunca apagados, setas de borda (periféricos) vermelhas para sempre, clock fixo em 10 ns em vez de vir do `platform.cfg`.
- Teste offscreen: passo a passo, play, modo rápido até o fim (3101 pacotes, 62,5 ms), volta no tempo e reset sem erros.
- Commit `8fe9e6b` (16:36) e documentação completa em `documentation/implementacoes.md`, com 12 seções (`9196207`, 17:14).

Modo escuro (17:55)

- Problemas vistos nos prints: percentuais com fundo branco só no tema claro inicial, bloco branco na grade no modo escuro (os PNGs das setas não têm transparência) e o rótulo `00.00%` da porta local sobre a borda do roteador.
- Correção: o fundo branco dos PNGs vira transparência ao carregar, as cores vêm do tema e o rótulo foi reposicionado. Commit `39e16ea`.

Communication Overview, Task Mapping e Message Log (18:08–18:21)

- As duas janelas passaram a acompanhar a simulação (no Java eram uma foto do momento em que abriam). Mapa de calor de azul (0%) a vermelho (100%), filtros por serviço e por Global, All traffic ou Router.

- Bug encontrado no teste: no modo escuro todas as células ficavam com o mesmo azul. Corrigido.
- Message Log: linhas `---->>>` (envio) e `<<<----` (recebimento). Teste: 2755 mensagens no fim, o mesmo número calculado direto dos pacotes.
- Commit `c3112c9`.

Deloream (18:26)

- Portado do pacote `deloream` do Java. Lê `log/log*.txt` do cenário e procura linhas `$$$_<processador>_<app>_<tarefa>_<mensagem>`. Duplo clique numa tarefa mostra as mensagens por processador; ex.: `prod[769]` mostra "Inicio da aplicacao prod" e "Fim da aplicacao prod".
- Bugs do Java corrigidos: o `_` dentro das mensagens era perdido e o Exit do Deloream fechava o programa inteiro. Commit `a72ee03`.

Executável e release alpha (18:35–19:26)

- Executável Linux gerado com o mesmo comando PyInstaller do CI (`dist/MemphisVDebugger/`, cerca de 180 MB).
- Falha: rodei o executável de `build/` e deu `Failed to load Python shared library … libpython3.14.so.1.0`. Causa: `build/` só tem arquivos temporários; o certo é `dist/`.
- Build no Windows: PyInstaller não faz build cruzado, então fiz em uma máquina Windows. Duas falhas no caminho: `uv` não reconhecido (PATH só vale em terminais novos) e `Compress-Archive` bloqueado porque o `.exe` ainda estava aberto.
- Release `v0.1.0-alpha` (pre-release) com o que já funcionava e o que faltava, com executáveis Linux e Windows. Integrações na `main` pelos PRs #2, #3 e #4.

Próximos passos:

- ~~Fazer a simulação do tráfego sobre a grade.~~
- Janela de informações do roteador (clique no roteador).
- Filtro de serviços e PEs, Platform Setup, projetos (salvar/abrir/apagar).
- Preparar a apresentação do estado do projeto.

25/09/26

O port ficou completo em relação ao Java, foram implementadas funções sugeridas pelo artigo do Memphis e foi publicada a versão 1.0.0. Também foram feitos os dois experimentos de validação (H1 e H2).

Apresentação (08:18)

- Pedido: um PowerPoint explicando o funcionamento, o que já foi refatorado e o que falta, com roteiro.
- Criados `documentation/apresentacao_memphis_debugger.pptx` (15 slides, com capturas reais do sistema e a fala nas notas) e `documentation/roteiro_apresentacao.md` (cerca de 20 min, duas demonstrações ao vivo e perguntas prováveis).

Experimento: a mesma simulação dá resultados diferentes no Java e no Python? (09:16)

- Observação: comparando prints do fim da simulação, o Java mostrava setas vermelhas que o Python não mostrava, e a diferença crescia tick a tick.
- Método: emular a lógica Java pacote a pacote (com `float` de 32 bits via `struct`) e comparar com a Python nos 3101 pacotes do exemplo.

O que foi comparado	Resultado
Recálculos do % dos enlaces	Iguais: 101 recálculos, nos mesmos pacotes
Percentuais, tempo e pacote atual	Iguais: 62,51176 ms, último pacote 0x0 → 0x0 MESSAGE_DELIVERY, 00.03% no 0x0
Setas vermelhas	Diferentes a partir do 3º pacote (tick 43490), em 3092 dos 3101 pacotes

- Análise: o Java decide se apaga o caminho por um contador de "pacotes em trânsito" que sobe na porta LOCAL0 e só desce na LOCAL1. No fim do exemplo ele está em 1356, então os caminhos se acumulam (até 106 setas ao mesmo tempo). O reset do Java também não apaga setas de borda; a emulação previu exatamente as duas setas do print (SOUTH do 0x0 e NORTH do 2x2).
- Conclusão: a diferença é correção de bug, não erro do Python. H1 confirmada.

Router Information (09:25)

- Pedido: implementar a janela do roteador como no Java (`RouterInfoFrame`). Novo `router_info_window.py`, aberto com clique no roteador.
- Abas Log, Applications, Scheduling e Traffic, janela Task Information e gráfico de escalonamento (verde para o kernel, azul para as tarefas). Lê `scheduling_report.txt`, `pipe/`, `request/` e `log/`.

Funções restantes do Java (13:08–13:25)

- Edit → Platform Setup: Flit Size, Clock Period e Window Width (no exemplo, 32, 10 e 500).

- Filters → Service and PE Filter (Ctrl+F): listas Only/Except por serviço e filtro por PE, com confirmação como no Java.
- File → Save/Open/Delete Project (`.hdf`), About, Packet Format e Print Router Total Link Usage. Corrigido o texto "Rest Graphical Path". Diferença: o Save pergunta antes de sobrescrever (o Java não perguntava).
- Commit `3f5a61f` (13:23). Revisão de tudo o que existia no Java: falta só abrir um debug pela linha de comando.

Funções sugeridas pelo artigo (13:43–15:18)

- Pergunta: o que o artigo de Ruaro et al. (DAES 2019) descreve para o debugger e ainda não existe? Tudo o que a Seção 5 e a Figura 9 mostram já estava portado; faltavam filtro por aplicação, validação de roteamento e deadlock, ligação entre escalonamento e pacotes e as métricas da Seção 6.
- Implementados (novos `analysis.py` e `statistics_windows.py`):
  - filtro por aplicação no Service and PE Filter e no Message Log (aplicação 2: 405 pacotes, Message Log de 2755 para 400 mensagens);
  - Tools → Traffic Statistics: vazão em Mbps por enlace (enlace de 3200 Mbps: 32 bits a 100 MHz), taxa de injeção e latência por aplicação;
  - Tools → Application Timeline: fases de mapeamento, alocação e execução. Admissão de 62 mil a 108 mil ciclos no exemplo (o artigo cita cerca de 40 mil para o MPEG);
  - abas Routing (validação XY) e Blocked (pacotes parados e espera circular). No exemplo: nenhuma violação XY em 1607 registros e nenhum pacote parado. Os serviços 0 a 3 do canal LOW não seguem XY, provavelmente a rede de broadcast de gerência;
  - exportação CSV/PNG (`export.py`) e File → Save Mesh as PNG;
  - caminho completo do pacote pintado na malha com duplo clique, pacotes de cada trecho do gráfico de escalonamento, periféricos lidos do YAML do testcase e Follow Live Trace (acompanha um trace ainda sendo gravado).
- Falha e correção na latência: a primeira versão casava cada chegada com a injeção mais recente e deixava 429 `MESSAGE_REQUEST` sem entrega. Descoberta: no trace do Memphis-V os campos de tarefa mudam entre os hops de um mesmo pacote. Com o rastreamento hop a hop:

Métrica	Primeira versão	Versão corrigida
Mensagens entregues	1136 de 1398	1388 de 1388
Latência mediana	~560 ciclos	7 ciclos

Métrica	Primeira versão	Versão corrigida
Latência máxima	825 mil ciclos	13.483 ciclos

- Os 10 registros removidos são `TASK_TERMINATED` com tamanho 0 e destino -1: avisos do kernel, não pacotes.
- Testes automatizados com pytest: 67 testes em cerca de 4 s, adicionados ao CI antes do build.
- Pergunta sobre banco de dados (como o do artigo): adiado. No artigo o banco fica no simulador, e o Memphis-V grava arquivos de texto. Um cache SQLite no debugger só compensa com traces grandes; antes é preciso medir com cerca de 2 milhões de pacotes.

Malha vetorial e Deloream com abas (16:58–17:14)

- Pedido: trocar as imagens dos roteadores e setas por desenho nativo do Qt, com zoom e periféricos fora do roteador. A malha virou `QGraphicsView` / `QGraphicsScene`; conferida em 93% e 220% e numa malha 4×4 com clusters 2×2.
- Deloream com abas e abertura com um clique; clicar numa aplicação abre as mensagens de todas as suas tarefas. 69 testes passando.

Testcase de estresse no Memphis-V (17:18–19:28)

- Pedido: com base em Memphis-V/regression, criar `tc.yaml`, `sc.yaml` e `app.yaml` que levem o sistema ao limite.
- `stress_6x6`: malha 6×6, 2 tarefas por PE (72 vagas, 69 livres), 117 tarefas em duas ondas (cerca de 1,7 vez a capacidade), logs TRAFFIC, UART, SCHED e PIPE ligados.
- Execução no servidor do laboratório: falhou com `No module named 'testcase'` (rodei uma cópia do `memphi5`) e com argumentos faltando (faltava a pasta `tc`).
- A simulação parecia travada depois de "Injection of prod finished". Análise: o TaskParser só imprime nas injeções e a simulação só termina quando todas as aplicações terminam; a onda 2 fica na fila do `mapper_task`. Não foi possível provar, porque a pasta de saída não foi encontrada.
- Criado `stress_6x6_small`: mesma malha, 69 tarefas, tudo cabendo na malha de uma vez.

Versão 1.0.0 (18:07–18:19)

- Commit `9ae2e9a` com 30 arquivos (cerca de 4.200 linhas), integrado pelo PR #5.
- Versão 1.0.0 no `pyproject.toml` (`e55ad1d`), PR #6 com CI verde (testes e build) e release v1.0.0, só com o executável Linux.

Experimento: o Python é mais rápido que o Java? (20:17)

- Método: `traffic_router.txt` do exemplo repetido 300 vezes (930.300 pacotes), processado sem interface pelo `getNextPacket` do `Memphis_Debugger.jar` e pelo `get_next_packet` do Python.

Versão	Tempo para 930.300 pacotes
Java (jar original)	7,18 s
Python	7,07 s

- Análise: empate na leitura (H2 confirmada). O desenho da interface não foi medido. A primeira execução do benchmark Java travou porque o processo Java não termina sozinho; foi refeita com contador de progresso.
- Ganhos qualitativos: `QTimer` em vez de thread com `sleep()`, redesenho só quando chega pacote, 200 pacotes por passo no modo rápido, seis bugs do Java corrigidos e as funções novas acima.

Próximos passos:

- ~~Janela de informações do roteador.~~
- ~~Filtro de serviços e PEs, Platform Setup, projetos.~~
- ~~Preparar a apresentação.~~
- Build Windows da 1.0.0.
- Abrir um debug pela linha de comando.
- Medir o desempenho com a interface desenhando.

28/09/26

Reduzido o número de arquivos Python de 31 para 21, sem mudar a lógica (H4 confirmada).

- Pergunta feita ao assistente, pedindo para eu decidir antes: "É possível diminuir a quantidade de arquivos Python sem afetar o funcionamento e desempenho do sistema?"
- Método: rodar os 69 testes antes, unir os módulos movendo as classes sem alterá-las, rodar os testes de novo.

Antes	Depois
`information/` (8 arquivos)	`information.py`
`util/MPSoCConfig.py`	`MPSoCConfig.py` na raiz (pasta `util/` removida)

Antes	Depois
`pe_matrix.py` + `slave_matrix.py`	`overview_matrices.py`
`platform_setup.py` + `help_dialogs.py`	`dialogs.py`
`Roteador.py`	removido (nenhum arquivo o importava)

- Ficaram separados `path_view.py` (unir com `main.py` criaria import circular) e `projects.py` (usado direto pelos testes).
- A estimativa inicial (17 a 19 arquivos) estava errada porque a contagem de partida era 31, não 30.
- Resultado: 69 de 69 testes passando e todos os módulos importando. Commit `58c8bec` (31 arquivos, +866/−1083 linhas), enviado para a `Develop`.
- O executável `MemphisVDebugger-linux-x86_64.tar.gz` (75 MB) ficou fora do commit, por ser gerado.

29/09/26

Integração com o menu global do GNOME (extensão Orbit Global Menu); funciona, mas ainda não foi commitada.

- Problema: no Wayland o Qt não exporta a barra de menus para a extensão. Solução: novo `global_menu.py` dá ao app o identificador `memphis-v-debugger` e gera, a partir da barra de menus real, o arquivo de mapeamento da extensão (cada item vira um atalho de teclado).
- Falha observada: o menu funcionava e depois parava, e o painel mostrava "python" com um menu genérico.
- Investigação: rastreando o protocolo Wayland, a janela principal sempre enviava `memphis-v-debugger`; o diálogo de arquivos (uma janela GTK criada pelo Qt) enviava `python`, e o GNOME passava a agrupar o app inteiro como "python".
- Correção: chamar `g_set_prgname` antes de criar a `QApplication`. Verificado no rastreamento do protocolo; falta confirmar no painel abrindo e fechando o diálogo.

Próximos passos:

- Confirmar no painel do GNOME a correção do menu global e commitar `global_menu.py`.
- ~~Integrar a `Develop` na `main`~~. Feito no PR #7, às 14:28 do mesmo dia, antes desta sessão.

Outubro/26 (data a confirmar) — pitch

Apresentado o pitch "Memphis-V Graphical Debugger: de Java para Python" (7 slides), com a pergunta de pesquisa da etapa seguinte.

- Problema: todos os elementos do MPSoC gravam no mesmo arquivo de log; em simulações grandes são muitos megabytes, caros de ler e filtrar. O banco de dados entre a extração de dados e a interface, citado no artigo, nunca foi implementado.
- Pergunta: como tratar e armazenar eventos de simulação para que o debugger escale com o tamanho do many-core?
- Proposta em três etapas: armazenar (estrutura indexada por PE e por tempo), consultar (a interface busca só o que está visível) e medir (disco, tempo de carga, filtros e memória em testcases crescentes). Apresentadas como hipóteses, sem resultados.

13/10/26

Max criou o workflow de release (`.github/workflows/release.yml`), que gera os executáveis de Linux (`.tar.gz`) e Windows (`.zip`) no GitHub Actions a cada tag.

- 19:29 — Primeira versão do pipeline (`eaf7f38`), testada com a tag `v1.0.1-BinaryTest` .
- 19:54 e 19:59 — Duas correções: ambiente (`4d37d20`) e empacotamento (`560ca47`), testadas com a tag `v1.0.2-BinaryTest` .
- Resultado: o build de Windows, que antes exigia uma máquina Windows, passou a ser automático.

14/10/26

Workflow de release integrado na `main` pelo PR #8 (`0ae2c8c`).

12/11/26

Preparação da apresentação final, com o Claude Code.

- Montado este caderno a partir do histórico do Git e das 9 sessões de desenvolvimento; versão em PDF em `documentation/Caderno_de_Pesquisa_Memphis-V_Debugger.pdf` .
- Rodado o debugger sem tela com o cenário de exemplo para tirar capturas reais. A suíte continua verde: 69 casos de teste em 5,4 s.
- Criada a apresentação final de 12 minutos (`documentation/Apresentacao_Final_Memphis-V_Debugger.pptx`), no visual do pitch, com as hipóteses H1–H4 e seus resultados, e o roteiro dividido entre os quatro

integrantes.

Próximos passos:

- Ensaiar a apresentação com o tempo de cada integrante.
- Começar a etapa "medir" da proposta do pitch.

Artefatos

Todo o código e a documentação estão no repositório do projeto; os testcases de estresse ficaram fora dele.

Artefato	Onde acessar
Código-fonte (21 arquivos Python, cerca de 7.900 linhas)	github.com/DrakRosmann/ProjectMemphisPy
Release v1.0.0	releases/tag/v1.0.0
Releases de teste do pipeline (Linux e Windows)	Tags `v1.0.1-BinaryTest` e `v1.0.2-BinaryTest`
Release v0.1.0-alpha (Linux e Windows)	Página de releases do repositório
Documentação técnica do port	`documentation/implementacoes.md`
Apresentação da v0.1.0-alpha e roteiro	`documentation/apresentacao_memphis_debugger.pptx`, `documentation/roteiro_apresentacao.md`
Pitch (outubro/26)	`Pitch_Memphis-V_Debugger.pptx` (fora do repositório)
Apresentação final (12 min)	`documentation/Apresentacao_Final_Memphis-V_Debugger.pptx`
Este caderno em PDF	`documentation/Caderno_de_Pesquisa_Memphis-V_Debugger.pdf`
Testes automatizados (58 testes, 69 casos)	`tests/`
CI: testes e build em PRs; executáveis em tags	`.github/workflows/build.yml`, `.github/workflows/release.yml`
Cenário de exemplo (9 PEs, 3101 pacotes, 11 tarefas)	`DEBUG EXAMPLE.zip`

Artefato	Onde acessar
Testcases `stress_6x6` e `stress_6x6_small`	Computador local e servidor do laboratório; não versionados
Debugger original em Java	github.com/gaph-pucrs/GraphicalDebugger
Framework Memphis-V e testes de regressão	github.com/gaph-pucrs/Memphis-V
Artigo de referência	Ruaro et al., *Memphis: a framework for heterogeneous many-core SoCs generation and validation*, Design Automation for Embedded Systems, 2019

Pendências

Seis itens continuam abertos em 12/11/26; o build de Windows foi resolvido pelo CI em 13/10/26.

- [ ] Confirmar no painel do GNOME a correção do menu global e commitar `global_menu.py`.
- [ ] Responder à pergunta do pitch: medir memória, tempo de carga, Back To e filtros com um trace de cerca de 2 milhões de pacotes, e só então decidir sobre o armazenamento indexado.
- [ ] Abrir um debug pela linha de comando (`main.py <platform.cfg ou pasta debug/>`), único item do Java ainda não portado.
- [ ] Medir o desempenho com a interface desenhando.
- [ ] Rodar o `stress_6x6_small` no servidor e abrir o resultado no debugger (teste do Follow Live Trace com simulador real).
- [ ] Adicionar ao `.gitignore` o `MemphisVDebugger-linux-x86_64.tar.gz` e remover o `Roteador.ui`, que não é mais usado.
- [x] ~~Gerar o build Windows da v1.0.0.~~
  Resolvido pelo workflow de release em 13/10/26.

Questão em aberto: as reuniões com o orientador e a data exata do pitch não aparecem no histórico do Git nem nas sessões; completar à mão.